# 遙測與空間資訊之分析與應用 - Week 2

國家災害防救科技中心 資訊組
台灣大學土木工程學系 教授
蘇文瑞
wraysu@ntu.edu.tw

# The 20-Meter Life-and-Death Gap

**"In disaster response, almost right is completely wrong."**

- Case Study: 0923 Mataian Creek Barrier Lake (馬太鞍溪堰塞湖)

  2025/9/23: Barrier lake breached, 15.4M tons of water in 30 min, 19 dead.

  Multi-agency response: evacuation of 1,800+ households across 3 townships required precise spatial coordination.

- The Coordinate Conflict:

  NCDR uses TWD97 (Meters); Google Maps uses WGS84 (Degrees).

  A CRS mismatch = drone surveying the wrong ridge, evacuation boundary shifted.

**When 8,000 people need evacuation, coordinate precision is life or death.**

# Recap: Week 1 Infrastructure

**What We Built Last Week:**

- Git/GitHub:

  gh CLI for repo management and version control.
- Environment:

  .env for API keys and global configs.
- Vibe Coding:

  Intent-driven development using Windsurf Cascade.

**Today's Goal: Moving from "Displaying Data" to "Auditing Data Truth."**

# The Physics of CRS: Why It Matters (坐標參考系統)

- The Earth is Not Flat:

  Ellipsoids (GRS80 vs. WGS84) - mathematical models of the Earth's shape.
- Datum (基準面):

  Where is the center of the earth? Different datums = different origins.
- Projections (投影):

  Turning a sphere into a flat map (TM2 in Taiwan).
- The Cost of Transformation:

  Every time you reproject, you risk a floating-point error.

# Taiwan's Spatial DNA: The EPSG Maze

- EPSG:4326 (WGS 84):

  Global GPS standard. Unit: Degrees. Best for: Web APIs, Global datasets.

- EPSG:3826 (TWD97 / TM2 zone 121):

  Taiwan's local engineering standard. Unit: Meters. Best for: Distance calculation, area estimation.

- EPSG:3857 (Web Mercator):

  Used by Google/Leaflet for visual browsing. Unit: Pseudo-meters.

  Why Google uses it: conformal (shape-preserving), simple tile math (2^n grid).

  Designed to "look right" for navigation, NOT to "calculate right" for analysis.

**WARNING: At 23.5°N, scale = 1/cos(23.5°) ≈ 1.091, area inflated ~19%. Never use for area calculation!**

# The "AI Mirage" in Spatial Coding

**Why AI Hallucinates Spatial Code:**

- AI training data is 90% Global (US/Europe).

  It defaults to EPSG:4326 or EPSG:3857.
- It rarely knows the specific shift between TWD67 and TWD97.

  Taiwan-specific parameters are underrepresented in training data.

**The Captain's Check:**

If the AI doesn't ask "What is your source CRS?", it is making a dangerous assumption.

# Logic Trap: set_crs() vs. to_crs()

- set_crs(3826):

  "This data IS in TWD97." (Assigning identity, no math performed).

- to_crs(4326):

  "Recalculate the numbers from Meters to Degrees." (Transforming nature).

**The Failure Mode:**

Students often use set_crs to convert data - this is WRONG.

**Visual Check:**

If your coordinates are [121.5, 25.0] and you set_crs(3826), your point lands near the Equator in the South China Sea.

# Why AI Fails at Data Cleaning

- The "Zero Island" Problem:
  AI ignores (0,0) points because they look like valid floats.
- X/Y Swapping:
  Mathematical: (x, y) vs. Geographical: (Lon, Lat).
  AI often guesses based on column alphabetical order.
- Scale Sensitivity:
  AI treats 121.5 and 121.50000001 as similar tokens.
  But in spatial analysis, that difference can be a meter.

# Professional Workflow: The Auditor Mindset

**Phase 1: Diagnosis (Range Check)**

- Is 120 < X < 122? Likely WGS84 Lon.

  Is 170,000 < X < 310,000? Likely TWD97 Easting.

**Phase 2: Standardization**

- Load from .env: PROJECT_CRS=4326.

  Single source of truth for all team members.

**Phase 3: Outlier Removal**

- "Is this point within Taiwan's Bounding Box?"

# Infrastructure: .env Management

**Never Hardcode EPSG:**

- Bad Practice:

  gdf.to_crs(epsg=3826)  # Hardcoded!

- Professional Practice:

  target_crs = os.getenv('TARGET_CRS')

  gdf.to_crs(epsg=target_crs)

**Why?**

Disaster models often switch between local (EOC) and international (UN) systems.

# The Art of Data Cleaning: Swapped Coordinates

**The Problem:**

- Mixed data sources (Line, SMS, Sensors) often produce swapped Lat/Lon.

**The AI Prompt Strategy:**

"Identify rows where abs(Lat) > abs(Lon). These are likely swapped. Fix them and log the row IDs."

**Log Everything:**

- A data scientist is a forensic investigator.

  Every transformation must be traceable.

# Exercise 1 : Shelter Data Audit

**Data Source:**

- 避難收容處所點位檔案 (data.gov.tw/dataset/73242)

  5,973 shelter locations across Taiwan. Real government data.

**Your Mission:**

- Download the CSV. Explore it. Find what’ s wrong.
- Ask AI to help you fix it. Observe where it succeeds — and fails.
- Write an audit report documenting every issue and correction.

**Rules:**

No hints. No answer key. You are the auditor.

# Reflection: What Did the AI Miss?

**After the Exercise, Discuss:**

- What types of errors did you find?
- What was your first prompt to AI? Did it work?
- Which errors did the AI catch? Which did it miss entirely?
- How did you refine your prompts to get better results?

**Deliverables (Push to GitHub):**

- audit_report.md — Issues found + corrections made
- ai_prompts.md — Every prompt you used + AI's response summary
- reflection.md — What you learned about AI's spatial blind spots

# Spatial Sanity Check: The "Golden Layer"

**Validation via Intersection:**

- Step 1: Load Taiwan County Boundary (Validated source).

  Step 2: Spatial Join: point within county.

  Step 3: Any point that doesn't join is an Outlier.

**AI Task:**

- "Write a script to visualize outliers in Red and valid points in Green."

  Always validate against authoritative geometry.

# Case Study: The 1999 Jiji Earthquake Data

**Context:**

- Historical data is often in TWD67 (EPSG:3821).

**The Shift:**

- TWD67 to TWD97 is approximately 800 meters.

  800 meters = the difference between a school and a riverbed.

**AI Hallucination:**

- AI might use a simple linear shift - this is WRONG.

**Professional Solution:**

- Use pyproj with official shift parameters from the Ministry of Interior.

# Exercise 2: CRS Compare Branch

## Step 1: Create a Branch

```
git checkout -b crs_compare
```

## Step 2: Give Cascade ONE Prompt

- 「氣象站 API 每個測站有兩組坐標，請都當成 WGS84 畫在同一張圖，並統計差距」

## Step 3: Commit, Push, Share

```
git add . && git commit -m "CRS compare analysis"
git push -u origin crs_compare
```

## Expected Result:

TWD67 vs WGS84 ≈ 850 meters apart. Same as Jiji's 800m!

# Show & Tell: Your CRS Map

**Present Your Findings (2 min each):**

- Show your comparison map — what does the offset look like?
- Share your distance statistics — average? max? min?
- What did Cascade do well? Where did it need your guidance?

**Discussion:**

- 850m ≈ 3 city blocks. What could go wrong in a disaster?
- If TWD67 and WGS84 differ by 850m, why does the API keep both?
- Why use a branch? (Answer: experiments stay isolated from main)

**Bonus Challenge:**

Use pyproj to properly convert TWD67 → WGS84. Does the gap disappear?

# Vibe Coding Strategy: Strategic Prompting

**Don't Ask:**

- "Clean this spatial data." (Too vague!)

**Do Ask:**

- "Act as a GIS Auditor."

  "1. Check the CRS of the input CSV."

  "2. Detect rows outside Taiwan bounds."

  "3. Ensure all outputs are EPSG:4326."

  "4. Generate a validation report in Markdown."

# The "Infrastructure First" Checklist

**Demo: Let Cascade Build Your Project Skeleton**

- Cascade Prompt:

  "Build a spatial data project with: .env (PROJECT_CRS=4326, Taiwan BOUNDING_BOX), requirements.txt (geopandas, pyproj, shapely), src/utils/spatial_helper.py (bounds-check function), and tests/ (verify to_crs conversion with known Taipei 101 coordinates)."

- What Cascade Generates:

  .env: CRS and bounding box config (no more hardcoding!)

  requirements.txt: dependency management

  spatial_helper.py: reusable audit functions

  tests/: auto-verify coordinate conversion (we'll learn this in Week 3)

**AI excels at scaffolding. Your job: verify the logic inside.**

# Demo Preview: The Faulty CSV

**We will load a CSV with:**

- Meters and Degrees mixed in the same file.
  Points in the ocean (invalid coordinates).
  Swapped Lat/Lon columns.

**The Exercise:**

- Watch Cascade (AI) try to fix it.
  Identify where the AI fails and where we must intervene.
  Document every correction in your audit log.

# Lab Task 1: Diagnostic Script

**Your Mission:**

- Analyze messy_data.csv.
  Find the hidden CRS without being told.
  Plot the bounding box to verify your detection.

**Deliverables:**

- A Python script that auto-detects CRS from coordinate ranges.
  A Markdown report explaining your reasoning.
  Push to GitHub with descriptive commit message.

# Lab Task 2: Multi-Layer Join

**Your Mission:**

- Align River Data (EPSG:3826) with Shelter Data (EPSG:4326).

  Calculate Euclidean distances between shelters and nearest rivers.

  Export results to GitHub.

**Key Challenge:**

- Both layers must be in the SAME CRS before distance calculation.

  Use .env to define the working CRS.

# Week 2 Reflection: The Pilot's Responsibility

**Thought Experiment:**

- If your AI-generated flood map causes an evacuation of the wrong village, who goes to court?

**Accountability:**

- Code is a suggestion; Logic is your responsibility.

**Growth:**

- From "Writing Code" to "Architecting Trust."

  The AI is your co-pilot, but you sign the flight plan.

# Conclusion & Next Week Preview

**Key Takeaways:**

- Spatial Integrity is the soul of GIS.

  AI is a powerful engine, but you are the Navigator.

  Always audit: CRS, bounds, coordinate order.

**Next Week: Vector Analysis & Heatmaps**

- Spatial joins at scale.

  Kernel Density Estimation for disaster hotspots.

  Interactive dashboards with Streamlit.

# Homework 2: Spatial Cross-Analysis (AQI × Shelters)

**情境：**颱風過後，哪些避難所位在空氣品質不良區域？

- **Task 1 — Data Audit:** 下載避難所 CSV，找出 ≥3 種坐標問題，產出 audit_report.md
- **Task 2 — Spatial Overlay:** AQI 測站 + 避難所疊圖（folium）
- **Task 3 — Nearest Station:** AQI > 100 標記高風險
- **Task 4 — Commit, Push, Reflect:** week2-shelter-analysis branch + reflection.md

**繳交：**Push to GitHub before next class

詳細說明請參考 Homework-Week2 講義

*"Code is a suggestion; Logic is your responsibility."*